



knjpnfnepek

```
int jump(int i, vector<int> &x) {
    int n=x.size();          Zabij skoki: rabimo inf,
    if (i==n-1) return 0;    a, b, mem_jump[]
    if (mem_jump[i]!=0) return mem_jump[i];
    int best=inf;
    for (int j=i+1; j<n; j++) {
        int dx[j]-x[i];
        if (a<d && d<=b) best=min(best, 1+jump(j,x));
    }
    mem_jump[i]=best;
    return best;
}
```

```
int partition(vector<int> a, int k) {
    int total=0, largest=0;
    for (int x : a) {
        total+=x;
        largest = max(largest, x);
    }
    // lo=feasible, hi=total;
    int lo=largest-1, hi=total;
    while (lo<hi) {
        int lim=(lo+hi)/2;
        int sum=0, chunks=1;
        for (int x : a) {
            // extend last chunk
            Enakomerno if (sum+x>lim) sum+=x;
            razbijte // start new chunk
            seznama else { chunks++; sum=x; }
            if (chunks<=k) hi=lim;
            else lo=lim;
        }
        return hi;
    }
}
```

```
VI toposort(vector<VI> &sosedi, int n) {
    vector<int> indeg(n);      Topološko
    for (int x=0; x<n; x++) { urejanje
        for (int y : sosedi[x]) indeg[y]++;
    }
    queue<int> q;
    for (int x=0; x<n; x++) {
        if (indeg[x]==0) q.push(x);
    }
    vector<int> seq;
    while (!q.empty()) {
        int x=q.front(); q.pop();
        seq.push_back(x);
        for (int y : sosedi[x]) {
            indeg[y]--;
            if (indeg[y]==0) q.push(y);
        }
    }
    return seq;
}
```

```
string LCS(string s, string t) {
    int n=s.size(), m=t.size();
    int lcs[n+1][m+1]; // dodatna vrstica in stolpec nicel
    memset(lcs, 0, sizeof(lcs));
    for (int i=n-1; i>=0; i--) {
        for (int j=m-1; j>=0; j--) {
            lcs[i][j]=max(lcs[i+1][j], lcs[i][j+1]);
            if (s[i]==t[j]) lcs[i][j]=max(lcs[i][j], 1+lcs[i+1][j+1]);
        }
    }
    // izpis izracunane tabele
    for (int i=0; i<n; i++) {
        for (int j=0; j<m; j++) {
            cout << lcs[i][j] << '\t';
        }
        cout << endl;
    }
    // rekonstrukcija
    string la="";
    int i=n, j=m;
    while (i>=0 && j>=0) {
        if (lcs[i][j]==lcs[i+1][j]) i++;
        else if (lcs[i][j]==lcs[i][j+1]) j++;
        else { la+=s[i]; i++; j++; }
    }
    return la;
}
```

```
void Dijkstra(vector<VI> &adjw, int start, vector<int> &dist, vector<int> &prev){
    int n=adjw.size();
    dist=vector<int>(n,-1); prev=vector<int>(n,-1);
    vector<int> p(n,-1); // provisional distance (-1=unvisited, -2=done)
    p[start]=0;
    while (1) {
        int x=-1; // smallest provisional
        for (int i=0; i<n; i++) if (p[i]>=0) {
            if (x==1 || p[i]<p[x]) x=i;
        }
        if (x==1) break;
        dist[x]=p[x]; p[x]=-2;
        for (auto [y,w] : adjw[x]) { // update neighbors
            int d=dist[x]+w;
            if (p[y]==-1 || (p[y]>=0 && d<p[y])) {
                p[y]=d; prev[y]=x;
            }
        }
    }
}
```